

Optimization Techniques

Lab n°5 - Linear Programming and Simplex Algorithm

In high dimension, the classical simplex algorithm may be very slow. This is due essentially to the matrix products $B^{-1}A$ and AM (see pages 32/39 of the lecture slides). The pseudocode here below presents a faster version of the simplex algorithm that was presented in the slides of the lecture slides. The main differences are

- We do not reshuffle the position of the columns at every step. In the first step, the basic variables are the first m . From Step 2, we save a vector (called *base_idx*) containing the indices of the basic variables.

Therefore, the vector $(c_1, \dots, c_m)$ used to compute the reduced cost is just substituted by $c_B = c(\text{base_idx})$

(for instance, if $\text{base_idx} = [3, 5, 2]$, then $c_B = (c_3, c_5, c_2)$)

- We do not apply the change of variables B^{-1} to the whole matrix A . To do so, we compute $y = c_B B^{-1}$ (line 7) and then $y \cdot A$ (line 8). Doing so, we always do a product *matrix* $\times$ *vector* (or *vector* $\times$ *matrix*) instead of *matrix* $\times$ *matrix*.

The Simplex algorithm

Algorithm 1 Simplex Algorithm (towards revised simplex algorithm)

Input: a constraint matrix $A \in \mathbb{R}^{m \times n}$, a vector $\mathbf{b} \in \mathbb{R}^m$, a cost vector $\mathbf{c} \in \mathbb{R}^n$. It assumes the initial basis corresponds to the first m columns.

```

1:  $base\_idx \leftarrow [1, 2, \dots, m]$  ▷ Array tracking the indices of variables in base
2:  $OptFound \leftarrow \text{False}$ 
3: while not  $OptFound$  do
4:    $B \leftarrow A(:, base\_idx)$  ▷ Extract current basic matrix from original A
5:    $B_{inv} \leftarrow B^{-1}$  ▷ Compute inverse
6:    $\mathbf{c}_B \leftarrow \mathbf{c}(base\_idx)$  ▷ Extract costs of current basic variables
7:    $\mathbf{y} \leftarrow \mathbf{c}_B \cdot B_{inv}$ 
8:    $\mathbf{r} \leftarrow \mathbf{c} - \mathbf{y} \cdot A$  ▷ Compute reduced costs vector
9:   if all  $r_j \geq 0$  then ▷ Optimality test
10:     $OptFound \leftarrow \text{True}$ 
11:     $\mathbf{x}_{opt} \leftarrow B_{inv} \cdot \mathbf{b}$  ▷ Calculate values for basic variables
12:    return  $\mathbf{x}_{opt}, \mathbf{c}_B \cdot \mathbf{x}_{opt}$ 
13:  end if
14:   $j_{en} \leftarrow \arg \min_i (r_i)$  ▷ Index of the entering variable
15:   $\tilde{\mathbf{a}}_{j_{en}} \leftarrow B_{inv} \cdot A(:, j_{en})$  ▷ Computes the coordinates of the column  $\mathbf{a}_j$  with respect to the basis of basic variables
16:   $\tilde{\mathbf{b}} \leftarrow B_{inv} \cdot \mathbf{b}$  ▷ Computes the coordinates of  $\mathbf{b}$  with respect to the new basis of basic variables
17:  if all  $(\tilde{\mathbf{a}}_{j_{en}})_i \leq 0$  then ▷ Unboundedness test
18:    return "Problem is unbounded"
19:  end if
20:   $Min\theta \leftarrow \infty$ 
21:   $q \leftarrow -1$ 
22:  for  $i = 1 \rightarrow m$  do ▷ Index of the variable that will leave the basis, initialized to -1
23:    if  $(\tilde{\mathbf{a}}_{j_{en}})_i > 0$  then ▷ Finds the minimal ratio  $\tilde{b}_i / (\tilde{\mathbf{a}}_{j_{en}})_i$ 
24:       $\theta \leftarrow \tilde{b}_i / (\tilde{\mathbf{a}}_{j_{en}})_i$ 
25:      if  $\theta < Min\theta$  then
26:         $Min\theta \leftarrow \theta$ 
27:         $q \leftarrow i$  ▷ The  $q$ -th basic variable will leave
28:      end if
29:    end if
30:  end for
31:   $base\_idx(q) \leftarrow j_{en}$  ▷ Update basis indices
32: end while

```

Exercise 1 The goal of this exercise is to solve the following LP programming problem

$$\min 5x_1 + 5x_2 + 3x_3 + 6x_4 + 4x_5 + x_6$$

subject to

$$\begin{cases} x_1 + x_4 = 8 \\ x_2 + x_5 = 5 \\ x_3 + x_6 = 2 \\ x_4 + x_5 + x_6 = 6 \\ x_i \geq 0 \quad \forall i. \end{cases}$$

The system is already in canonical normal form, with

$$A = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 \end{pmatrix} \quad b = \begin{pmatrix} 8 \\ 5 \\ 2 \\ 6 \end{pmatrix} \quad C = (5, 5, 3, 6, 4, 1).$$

Write a program that implements Simplex algorithm, and apply it to the problem.

(The solution is $x = (8, 1, 0, 0, 4, 2)$.)

Sometimes, it is difficult to find a basic feasible solution to start the algorithm. For this reason, one could apply the *two step Simplex algorithm*.

In order to do so, we define an auxiliary problem We introduce new **Artificial Variables** $y_1, \dots, y_m \geq 0$ and define a new LP:

$$\begin{aligned} \text{Minimize} \quad & y_1 + \dots + y_m \\ \text{Subject to} \quad & Ax + y = b \\ & x \geq 0, y \geq 0 \end{aligned}$$

The original LP problem has a Basic Feasible Solution if and only if the artificial problem has an Optimal Feasible Solution with cost=0.

Algorithm 2 Two-Phase Simplex: Phase 1 Initialization

Input: a constraint matrix $A \in \mathbb{R}^{m \times n}$, a vector $b \in \mathbb{R}^m$, a cost vector $c \in \mathbb{R}^n$.

```

1:  $A_{aug} \leftarrow [I_m, A]$                                 ▶ Augmentation of the matrix. Virtual variables BEFORE
2:  $c_{virt} \leftarrow [1_m, 0_n]$                             ▶ Objective function for Phase 1
3:  $\text{Phase1Out} \leftarrow \text{SimplexAlg}(A_{aug}, b, c_{virt})$ 
4:  $idx_{init} \leftarrow \text{Phase1Out}(:, 2)$                     ▶ Indexes of the basic variables selected by the first phase
5:  $A_{new} \leftarrow A_{aug}(:, idx_{init})$                     ▶ Initialize new matrix with basic columns
6:  $c_{new} \leftarrow c(idx_{init} - m)$                         ▶ Shift indices to retrieve original costs
7: for  $h = m + 1 \rightarrow m + n$  do
8:   if  $h \notin idx_{init}$  then
9:      $A_{new} \leftarrow [A_{new}, A_{aug}(:, h)]$                 ▶ Append non-basic columns
10:     $c_{new} \leftarrow [c_{new}, c(h - m)]$                 ▶ Append corresponding original costs
11:   end if
12: end for

```

Mind to transform the system into canonical form! Although A_{new} now has the basic variables in its first m columns, they do not form an identity matrix. To start Phase 2, you perform a coordinate change: set $CCh = A_{new}(:, 1 : m)$ and update the system as follows:

$$\begin{aligned} A_{new} &\leftarrow CCh^{-1} \cdot A_{new} \\ b &\leftarrow CCh^{-1} \cdot b \end{aligned}$$

This guarantees that your initial basis for Phase 2 is a perfect identity matrix.

Exercise 2 Modify your code to implement the two-phase simplex algorithm to solve the LP

$$\min -6x_1 - 4x_2 - 7x_3 - 5x_4$$

subject to

$$\begin{cases} x_1 + 2x_2 + x_3 + 2x_4 \leq 20 \\ 6x_1 + 5x_2 + 3x_3 + 2x_4 \leq 19 \\ 3x_1 + 4x_2 + 9x_3 + 12x_4 \leq 75 \\ x_i \geq 0 \quad \forall i. \end{cases}$$

The put the system in normal form, we must add the slack variables y_1, y_2, y_3 . Thus, the data of the problem in normal form are:

$$A = \begin{pmatrix} 1 & 2 & 1 & 2 & 1 & 0 & 0 \\ 6 & 5 & 3 & 2 & 0 & 1 & 0 \\ 3 & 4 & 9 & 12 & 0 & 0 & 1 \end{pmatrix} \quad b = \begin{pmatrix} 20 \\ 19 \\ 75 \end{pmatrix} \quad C = (-6, -4, -7, -5, 0, 0, 0).$$

(The solution is $x = (0, 0, 13/3, 3)$.)